

BUSINESS INTELLIGENCE

Rafael Lucas Oliveira Bergamin

Integração do Streamlit ao Data Warehouse (C3)

22 de junho de 2026

Sumário

1	Introdução	2
2	Objetivos do Projeto (C3)	2
3	Metodologia de Desenvolvimento	2
3.1	Etapa 1: Conclusão do Data Warehouse	2
3.2	Etapa 2: Finalização do Pipeline de ETL	3
3.3	Etapa 3: Refatoração do Dashboard (Streamlit)	3
3.4	Etapa 4: Avaliação do Benchmark	3
4	Conclusão da Análise de Desempenho	3

1 Introdução

O presente relatório documenta a fase final e integradora (C3) do projeto de análise de dados de COVID-19 do Espírito Santo. Nas etapas anteriores, desenvolveu-se uma camada visual autônoma fundamentada em arquivos tabulares (C1) e instituiu-se a modelagem teórica e inicial de um Data Warehouse em esquema estrela (C2). O objetivo central desta etapa é finalizar a engenharia de dados do banco analítico e conectar a camada de visualização em *Streamlit* para extrair os indicadores diretamente do SGBD. Por fim, o trabalho consolida uma análise de desempenho empírica contrastando as vantagens computacionais da modelagem dimensional frente à abordagem plana.

2 Objetivos do Projeto (C3)

O escopo principal deste projeto prático avaliativo visa concluir o ciclo de Business Intelligence através das seguintes entregas:

- **Conclusão do Data Warehouse:** Revisar o modelo dimensional e implementar pelo menos uma estratégia de Slowly Changing Dimension (SCD).
- **Finalização do ETL:** Refinar os processos de Extração, Transformação e Carga (ETL) no SGBD com tratamento de logs e registro de execução.
- **Nova conexão no Streamlit:** Adaptar o dashboard criado na etapa C1 para consumir e executar consultas SQL puras diretamente do Data Warehouse.
- **Análise de Desempenho:** Medir o impacto empírico do consumo de recursos (Tempo e Memória) comparando o modelo de planilhas locais (Parquet/CSV) com o modelo otimizado do Banco de Dados.

3 Metodologia de Desenvolvimento

O projeto foi dividido sistemicamente em 4 macro etapas executadas sobre o banco *SQLite* e a linguagem *Python*.

3.1 Etapa 1: Conclusão do Data Warehouse

A modelagem do DW, baseada na arquitetura de Kimball, foi mantida em um Esquema Estrela rigoroso composto por 1 Fato (FATO_NOTIFCOVID) e 7 dimensões (DIM_TEMPO, DIM_LOCALIDADE, DIM_PERFIL_PACIENTE, DIM_CLASSIFICACAO, DIM_SINTOMAS, DIM_COMORBIDADE, DIM_TESTE). Para cumprir o requisito de rastreabilidade temporal do negócio, aplicou-se a estratégia de **SCD Tipo 2** na dimensão *DIM_LOCALIDADE*.

Na implementação física (*01_create_schema.py*), o DDL foi alterado para incorporar as colunas de ciclo de vida do registro: *populacao_municipio*, *data_inicio*, *data_fim* e *flag_atual*. Isso garante que, caso a população de um município seja alterada, o registro antigo seja expirado (*data_fim* e *flag_atual* = 0) e um novo registro assuma a validade.

3.2 Etapa 2: Finalização do Pipeline de ETL

O fluxo de ETL foi unificado em um orquestrador central e automatizado intitulado *etl_final_c3.py*. Como prática de sustentação e engenharia de software profissional, aplicou-se a biblioteca nativa *logging*. Todas as execuções, transformações textuais (como inserção rigorosa de campos *-1* como "Desconhecido") e potenciais erros críticos foram gravados sistemicamente no arquivo de acompanhamento *etl_execution.log*. A carga de aproximadamente **5.186.537 registros** da base da SESA foi processada eficientemente para as tabelas fato em pouco menos de 3 minutos no banco relacional.

3.3 Etapa 3: Refatoração do Dashboard (Streamlit)

O aplicativo construído na C1 utilizava a biblioteca *Pandas* para alocar toda a base em um arquivo local e executar filtros por debaixo dos panos através das renderizações do Streamlit. Para o modelo da C3, desenvolveu-se o arquivo *app_dw.py*. O arquivo *Parquet* foi sumariamente descartado da lógica e os algoritmos visuais passaram a estabelecer uma conexão cliente-servidor com o *SQLite*.

Todos os 10 exercícios da tela (ex: Ranking de Cidades, Casos por Faixa Etária, Letalidade) foram reescritos em queries **SQL de alta complexidade**, forçando o Banco de Dados a executar os cruzamentos com a cláusula *JOIN* e retornar estritamente a consolidação dos dados. Aplicou-se também o decorador *@st.cache_data* para gravar no cache os milissegundos das queries.

3.4 Etapa 4: Avaliação do Benchmark

Foi programado o script independente *performance_test.py* com a finalidade exclusiva de realizar um teste de estresse de carga. O código mediu, comparativamente, os tempos de leitura e principalmente a escalada de memória RAM do sistema operacional ao resolver a consulta analítica (Agrupamento dos Top 10 Municípios) na versão com *Pandas/Parquet* contra a versão delegada em SQL.

4 Conclusão da Análise de Desempenho

A análise técnica coletou empiricamente os resultados de desempenho da aplicação e gerou as seguintes avaliações arquiteturais vitais para o negócio:

1. **Escalabilidade e Uso de Memória:** A abordagem legada (C1), ao carregar a tabela massiva em memória via *Pandas*, monopolizou impressionantes **5.657 MB (5.6 GB)** de RAM do servidor para alocar o *DataFrame*. Ao delegar o serviço de agregação ao SGBD na abordagem moderna (C3), o aplicativo Streamlit retornou o mesmo resultado exato utilizando ínfimos **0.0008 MB** de memória no processo cliente. Isso erradica o gargalo de OOM (*Out Of Memory*) e permite que a aplicação web rode nas infraestruturas mais enxutas possíveis.

2. **Latência do Usuário (Dashboard Load):** No cenário original (C1), o usuário final precisava aguardar quase 3 segundos inteiros de inicialização pesada toda vez que o servidor recarregasse a sessão, pois o *Pandas* transcrevia todo o binário para a RAM. Na C3, o SGBD realiza indexações rápidas. Embora uma query no DW possa demorar 1 ou 3 segundos devido à densidade de JOINS em 5 milhões de linhas, esse tempo ocorre estritamente na consulta solicitada e é imediatamente abatido pelo Cache do Streamlit, permitindo que a tela inicial abra instantaneamente.
3. **Viabilidade Operacional:** A separação de responsabilidades (Front-end no Streamlit ↔ Back-end analítico no Data Warehouse) garante a governança real dos dados. A introdução de chaves e esquemas garante integridade, impedindo a injeção acidental de dados inconsistentes que poderia ocorrer de forma arbitrária num CSV plano.

Em suma, a transição para a arquitetura Data Warehouse prova inquestionavelmente seu valor não apenas na capacidade analítica organizada, mas sobretudo como pilar fundamental de desempenho, governança, estabilidade computacional e escalabilidade profissional em sistemas de Business Intelligence modernos.